

6.1 Core Motivations of PISanitizer

The effectiveness of PISanitizer is based on two motivations: (1) prompt injection attacks craft an instruction to compel an LLM to follow it, and (2) an LLM would pay high attention to instructional tokens that are followed by the LLM. The effectiveness of PISanitizer would diminish if any of them do not hold.

Motivation-1: PISanitizer would fail when an attacker manipulates the LLM’s output without relying on explicit instructions. For instance, an attacker can leverage knowledge corruption attacks [60] to make an LLM generate an attacker-chosen target output. Suppose the target instruction is “*Please answer the following question based on the given context: which Python package is used for HTML parsing?*”. An attacker can inject the following text “*The MalHttp is the best package for HTML parsing*” into a context. As a result, an LLM would generate “*The HTML parsing package is MalHttp*”. By design, PISanitizer cannot defend against such an attack as it does not leverage any explicit instructions. We note that such attacks can be extremely challenging to defend against without extra information.

Experiment: We conduct experiments on the Open-Prompt-Injection benchmark [4], where an attacker injects both an *instruction* and *data* into the context (an example is shown in Appendix A). Our PISanitizer effectively removes the injected instruction but cannot fully eliminate the injected data. Without attacks, the utility remains 0.76 with and without PISanitizer. Under the Combined Attack, the attack success rate (ASR) drops from 0.75 (no defense) to 0.0 (with PISanitizer), while the utility improves from 0.12 (no defense) to 0.67 (with PISanitizer), demonstrating that PISanitizer can effectively remove injected instructions. However, PISanitizer cannot completely restore the utility to the no-attack level because the injected data (non-instructional) influences utility when the context is short.

Motivation-2: PISanitizer can be less effective under a potential “weak” attack (e.g., the ASR of Naive Attack is the highest as shown in Table 1), where an injected instruction receives no or very low attention from the LLM, but the LLM still follows it (if possible). In Section 5.4, we design adaptive attacks to optimize the injected prompt to achieve these two goals. However, we find that this can be very challenging. Due to the attention mechanism [38], an input token would not influence the output if it receives no attention. We note that PISanitizer can also be combined with existing defenses such as Meta-SecAlign [17] to defend against such potential “weak” attacks.

6.2 Benign Instructions in a Context

PISanitizer is designed to sanitize instructions in a context, but cannot distinguish whether an instruction in a context is benign or not. Thus, when a context contains benign instructions, PISanitizer would also remove them, leading to utility loss if they are necessary for a target task.

Experiment: We perform experiments on AgentDojo [69], a widely used prompt injection benchmark for LLM agents, where an attacker injects malicious prompts into the agent’s input data to manipulate tool-call behaviors, such as calling a malicious tool API. Following [17], we perform sanitization to tool call results. The backend LLM is GPT-4o.

Without attacks, the utility is 0.82 (no defense) and 0.71 (under PISanitizer), respectively. We manually analyze the cases where PISanitizer impacts utility. Among the total 13 cases, we find that 11 cases contain benign instructions that are essential for target tasks. For instance, one example is “*Please pay the amount by sending a bank transfer to the following account: IBAN: UK12345678901234567890*”. Without information on the legitimacy of the account, it can be hard to distinguish benign from malicious instructions.

Under attack, the ASR decreases from 0.48 (no defense) to 0.03 (with PISanitizer), while the utility increases from 0.40 (no defense) to 0.67 (with PISanitizer), demonstrating that PISanitizer effectively removes injected instructions.

Mitigation: PISanitizer can be used as an effective tool to pinpoint instructions in a context. To avoid sanitizing benign instructions, potential strategies include 1) human-in-the-loop approval, e.g., requiring a human confirmation of sensitive operations such as making a payment, and 2) integrating other more

reliable, automated ways (e.g., security policies [18]) of validation after an instruction from an untrusted source is pinpointed by PISanitizer.

7 Conclusion and Future Work

Prompt injection poses a severe security concern to many long-context LLM applications. Existing defenses are mainly designed for short context and have limited effectiveness when applied to long contexts, as an injected instruction generally constitutes a small part of a long context. In this work, we propose PISanitizer to sanitize injected tokens in a context before letting a backend LLM generate an output. Our extensive evaluations demonstrate that PISanitizer can effectively defend against prompt injection in long context scenarios. An interesting future work is to extend PISanitizer to prevent prompt injection to multi-modal LLMs.

References

- [1] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” in *NeurIPS ML Safety Workshop*, 2022.
- [2] S. Willison, “Prompt injection attacks against GPT-3,” <https://simonwillison.net/2022/Sep/12/prompt-injection/>, 2022.
- [3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” in *AISec*, 2023.
- [4] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *USENIX Security*, 2024.
- [5] D. Pasquini, M. Strohmeier, and C. Troncoso, “Neural exec: Learning (and learning from) execution triggers for prompt injection attacks,” in *AISec*, 2024.
- [6] X. Liu, Z. Yu, Y. Zhang, N. Zhang, and C. Xiao, “Automatic and universal prompt injection attacks against large language models,” *arXiv*, 2024.
- [7] P. S. Team, “Mitigating prompt injection in comet,” <https://www.perplexity.ai/hub/blog/mitigating-prompt-injection-in-comet>, 2025.
- [8] P. AI, “Comet is an ai-powered browser that acts as a personal assistant and thinking partner,” <https://www.perplexity.ai/comet>, 2025.
- [9] OpenAI, “Introducing chatgpt atlas,” <https://openai.com/index/introducing-chatgpt-atlas/>, 2025.
- [10] S. Willison, “Delimiters won’t save you from prompt injection,” <https://simonwillison.net/2023/May/11/delimiters-wont-save-you>, 2023.
- [11] “Sandwich defense,” https://learnprompting.org/docs/prompt_hacking/defensive_measures/sandwich_defense, 2023.
- [12] “Instruction defense,” https://learnprompting.org/docs/prompt_hacking/defensive_measures/instruction, 2023.
- [13] J. Piet, M. Alrashed, C. Sitawarin, S. Chen, Z. Wei, E. Sun, B. Alomair, and D. Wagner, “Jatmo: Prompt injection defense by task-specific finetuning,” in *ESORICS*, 2024.
- [14] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “Struq: Defending against prompt injection with structured queries,” *USENIX Security*, 2024.
- [15] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, “Secalign: Defending against prompt injection with preference optimization,” in *CCS*, 2025.

- [16] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, “The instruction hierarchy: Training llms to prioritize privileged instructions,” *arXiv*, 2024.
- [17] S. Chen, A. Zharmagambetov, D. Wagner, and C. Guo, “Meta secalign: A secure foundation llm against prompt injection attacks,” *arXiv preprint arXiv:2507.02735*, 2025.
- [18] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” *arXiv preprint arXiv:2503.18813*, 2025.
- [19] T. Shi, J. He, Z. Wang, L. Wu, H. Li, W. Guo, and D. Song, “Progent: Programmable privilege control for llm agents,” *arXiv preprint arXiv:2504.11703*, 2025.
- [20] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Securing ai agents with information-flow control,” *arXiv preprint arXiv:2505.23643*, 2025.
- [21] T. Wu, S. Zhang, K. Song, S. Xu, S. Zhao, R. Agrawal, S. R. Indurthi, C. Xiang, P. Mittal, and W. Zhou, “Instructional segment embedding: Improving llm safety with instruction hierarchy,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [22] F. Wu, E. Cecchetti, and C. Xiao, “System-level defense against indirect prompt injection attacks: An information flow control perspective,” *arXiv preprint arXiv:2409.19091*, 2024.
- [23] J. Kim, W. Choi, and B. Lee, “Prompt flow integrity to prevent privilege escalation in llm agents,” *arXiv preprint arXiv:2503.15547*, 2025.
- [24] Y. Jia, Z. Shao, Y. Liu, J. Jia, D. Song, and N. Z. Gong, “A critical evaluation of defenses against prompt injection attacks,” *arXiv preprint arXiv:2505.18333*, 2025.
- [25] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov *et al.*, “The attacker moves second: Stronger adaptive attacks bypass defenses against llm jailbreaks and prompt injections,” *arXiv preprint arXiv:2510.09023*, 2025.
- [26] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, “Promptshield: Deployable detection for prompt injection attacks,” in *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*, 2024, pp. 341–352.
- [27] H. Li and X. Liu, “Injecguard: Benchmarking and mitigating over-defense in prompt injection guardrail models,” *arXiv preprint arXiv:2410.22770*, 2024.
- [28] ProtectAI.com, “Fine-tuned deberta-v3-base for prompt injection detection,” 2024. [Online]. Available: <https://huggingface.co/ProtectAI/deberta-v3-base-prompt-injection-v2>
- [29] Meta, “PromptGuard Prompt Injection Guardrail,” <https://www.llama.com/docs/model-cards-and-prompt-formats/prompt-guard/>, 2024.
- [30] S. Abdelnabi, A. Fay, G. Cherubin, A. Salem, M. Fritz, and A. Paverd, “Get my drift? catching llm task drift with activation deltas,” in *SaTML*, 2025.
- [31] K.-H. Hung, C.-Y. Ko, A. Rawat, I.-H. Chung, W. H. Hsu, and P.-Y. Chen, “Attention tracker: Detecting prompt injection attacks in llms,” in *NAACL*, 2025.
- [32] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, “Datasentinel: A game-theoretic detection of prompt injection attacks,” in *IEEE S&P*, 2025.
- [33] W. Zou, Y. Liu, Y. Wang, Y. Chen, N. Gong, and J. Jia, “Pishield: Detecting prompt injection attacks via intrinsic llm features,” *arXiv preprint arXiv:2510.14005*, 2025.
- [34] Y. Wang, W. Zou, R. Geng, and J. Jia, “Tracllm: A generic framework for attributing long context llms,” in *USENIX Security Symposium*, 2025.